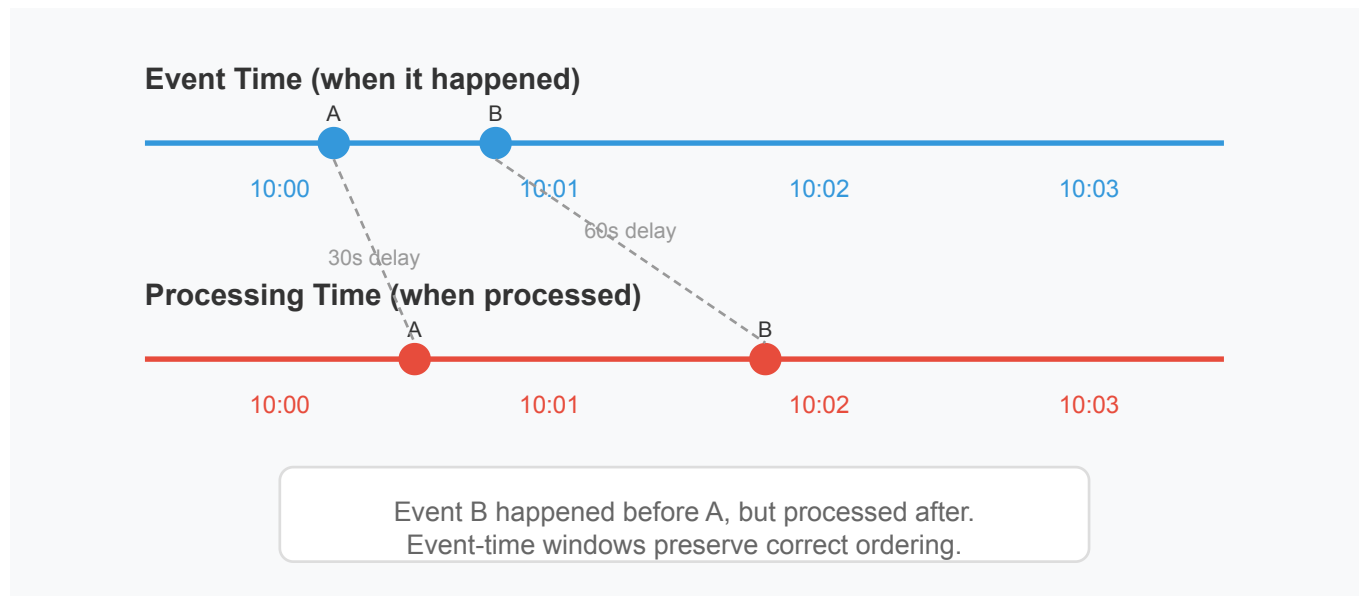


Event Time, Processing Time, and Watermarks in Stream Processing

 systemdesignschool.io/fundamentals/event-time-watermarks



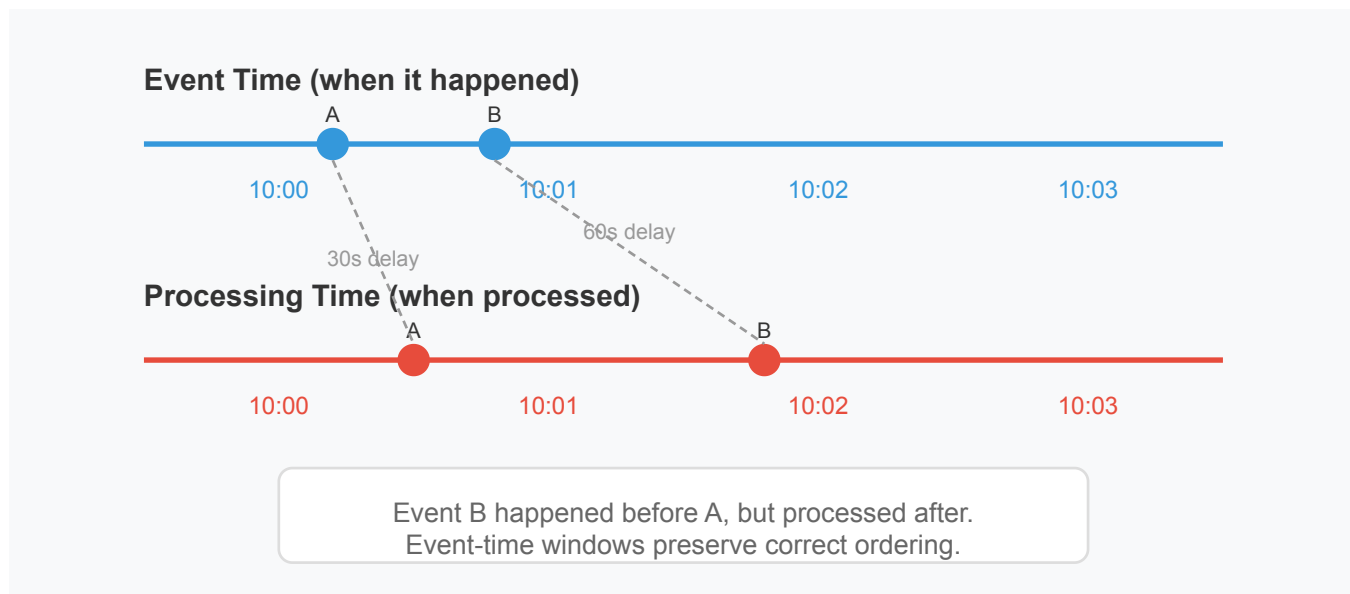
Event Time & Watermarks

The previous article showed how windows aggregate unbounded streams. But windows raise a question: when should they close? That depends on understanding two different notions of "when" an event happens.

Event Time vs Processing Time

Event time is when the event actually occurred—the timestamp recorded at the source. A user clicked a button at 10:00:00 according to their device. That timestamp is embedded in the event.

Processing time is when your system processes the event. Network delays, queue backlogs, or consumer restarts mean processing might happen at 10:00:05—or 10:05:00 if the system fell behind during an outage.



These two times can differ significantly. A mobile app might buffer events during spotty connectivity and send them in a batch when the connection recovers. Events from IoT devices in remote locations might take minutes to reach your datacenter.

Why the Difference Matters

Consider counting clicks per minute for a real-time dashboard.

With processing-time windows: You group clicks by when your system processes them. If a traffic spike creates a 5-minute backlog, clicks that happened at 10:00 get processed at 10:05 and count toward the 10:05 window. If a consumer crashes and restarts, replayed events might get double-counted (depending on your checkpointing). The same underlying data produces different results depending on system timing.

With event-time windows: Clicks count based on when users clicked—the timestamp in the event—regardless of processing delays. The same data always produces the same counts. This is called **deterministic reprocessing**: you can replay the input and get identical output.

For analytics that need to be reproducible, event-time windowing is essential.

The Out-of-Order Problem

Event-time processing introduces a challenge: events don't arrive in timestamp order.

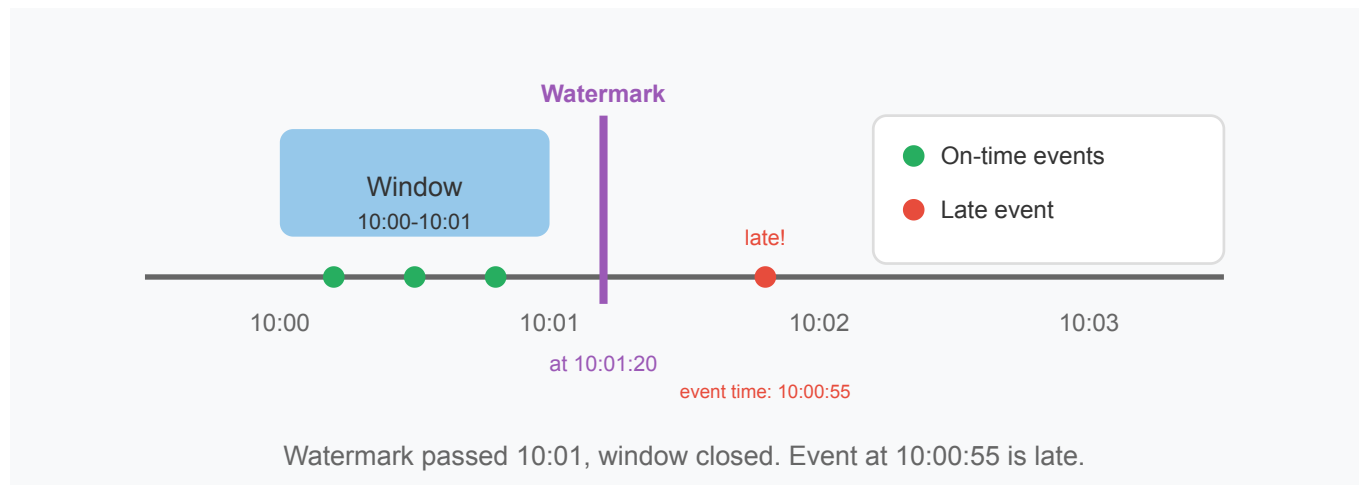
Suppose events from two different mobile users arrive at your system. User A's event from 10:00:55 travels through a slow cell tower. User B's event from 10:01:02 travels through fast wifi. You see 10:01:02 first, then 10:00:55.

If you close the 10:00–10:01 window as soon as you see a 10:01 event, you'll miss the 10:00:55 event. But you can't wait forever either—you need to emit results eventually.

Watermarks: Tracking Progress

Watermarks solve this by tracking the system's estimate of event-time progress. A watermark at time T means: "We believe all events with timestamps before T have arrived."

When the watermark advances past a window's end time, the system can close that window and emit results.



Watermarks are estimates, not guarantees. Late-arriving events can still appear after the watermark has passed their timestamp. The system must have a policy for handling them.

How Watermarks Are Generated

Different approaches suit different scenarios:

Perfect watermarks are possible when you control the source and know all events have been seen. For example, if you're reading from a bounded log file in order, the watermark can track exactly which timestamps have been fully consumed.

Heuristic watermarks estimate based on observed patterns. If you observe that 99% of events arrive within 30 seconds of their event time, you might set the watermark to lag 30 seconds behind the most recent event timestamp. This balances completeness against latency.

Source-based watermarks track progress from upstream systems. If the input is partitioned (like Kafka partitions), you track the minimum event-time progress across all partitions—the slowest partition bounds the watermark.

Handling Late Events

Events arriving after their window's watermark has passed are called **late events**. Common strategies:

Drop late events. Simplest implementation. Accept some data loss in exchange for finalized windows that never change. Works when approximate counts are acceptable.

Allowed lateness. Keep windows open for a grace period after the watermark passes. Events within the grace period update the window; events beyond it get dropped. Balances correctness against state retention.

Emit updates. When late events arrive, recompute and emit an updated result for the affected window. Downstream consumers must handle updates (retractions and new values). This is most correct but adds complexity.

Side output. Route late events to a separate stream for special handling—perhaps manual investigation or delayed reconciliation.

The Latency vs Completeness Trade-off

This is a fundamental tension in stream processing:

- **Aggressive watermarks** (advancing quickly, short allowed lateness) emit results sooner but may miss late events.
- **Conservative watermarks** (lagging further behind, longer allowed lateness) produce more complete results but add latency and increase state.

The right choice depends on your use case:

- A **real-time dashboard** showing active users can tolerate missing a few late events. Fast watermarks keep the display current.
- A **billing system** calculating charges needs completeness—it's better to wait and get the right answer than to undercharge and reconcile later.
- A **fraud detection system** needs speed for blocking transactions, so it might use aggressive watermarks for real-time scoring while running a separate batch reconciliation for accuracy.

Summary

Event time gives you deterministic, reproducible results regardless of processing delays. Watermarks let you make progress on event time even when events arrive out of order. The trade-off between latency and completeness is unavoidable—but at least you can make an explicit choice rather than having processing delays randomly distort your results.

That connects to **delivery guarantees**—what happens to events when processing nodes fail, and how to ensure events are processed exactly once (or at least understand when that's not achievable).